Repetition Structures
While loops

CSCI-SHU 11
Fall 2025

Lecture's Outline

- While loops
  - loops vs iteration
  - sentinel and accumulator variables
  - augmented assignment operators
- Modules
  - importing modules
  - calling a function from an imported module

Loops

- Write a program to print out the numbers 1 through 9

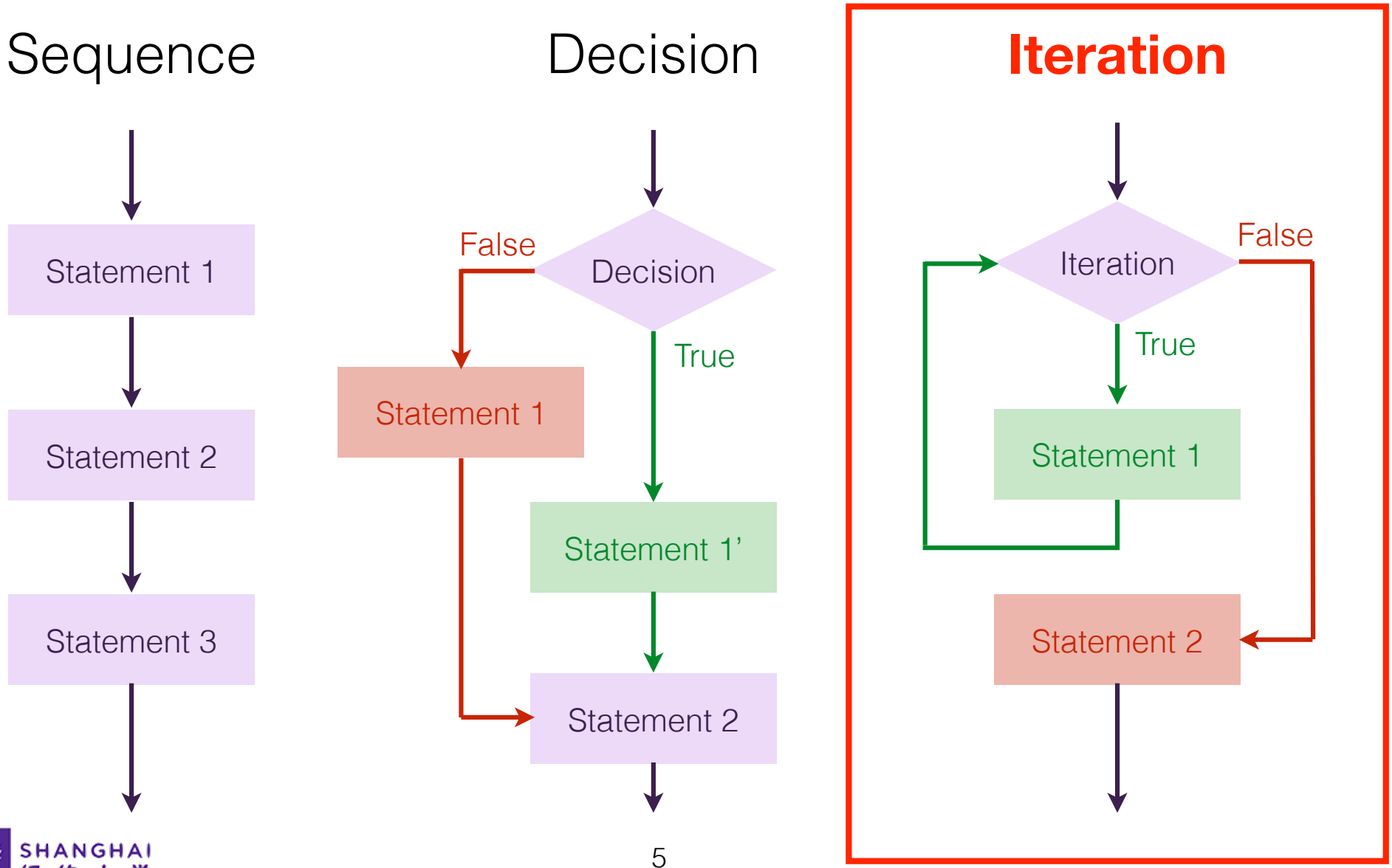
```
# 01-print_counter.py

n = 1     # n is 1
print(n)
n = n + 1      # n is 2
print(n)
n = n + 1      # n is 3
print(n)
n = n + 1      # n is 4
print(n)
n = n + 1      # n is 5
print(n)
n = n + 1      # n is 6
print(n)
n = n + 1      # n is 7
print(n)
n = n + 1      # n is 8
print(n)
n = n + 1      # n is 9
print(n)
```

```
1
2
3
4
5
6
7
8
9
```

Loops

- Is there a better solution?
  - Previous solution uses Sequence Structure
  - Previous solution is relatively long to code (even with copy-paste)
  - The chance for error is big
  - The code is not very readable (even with comments)
  - How do I manage if I want to print out the numbers 1 through 1,000,000 (or more)?
- A better solution is the use of Repetition Structures
  - *while* loops (today)
  - *for* loops (next lecture)

Control Structures

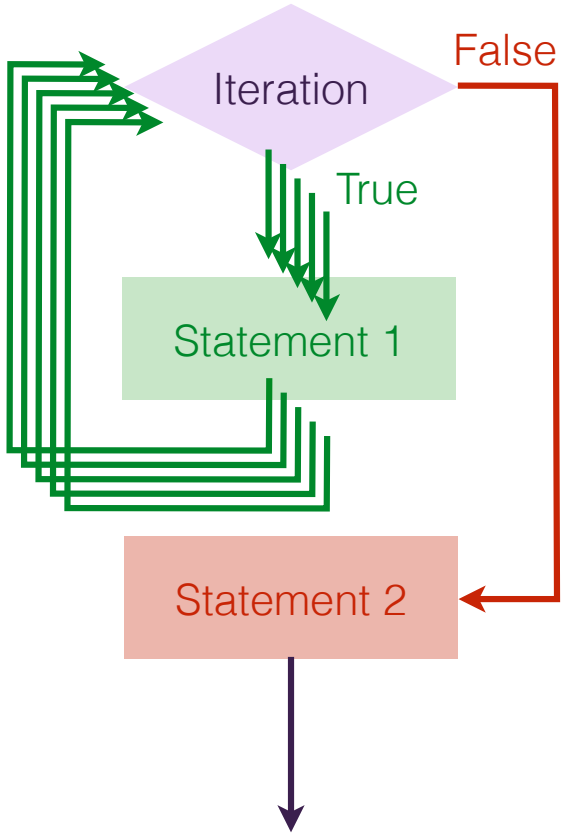


Iteration vs Loops

- Iteration
  - repeated execution of a set of programming statements
- Loop
  - the construct that allows us to repeatedly execute a statement or a group of statements until a terminating condition is satisfied
- Sometimes these words are used interchangeably
  - A single loop may consist of numerous iterations!

While Loops

- A loop is a condition-controlled loop
  - repeat a block of code…
  - …as long as a condition is True
  - stop as soon as the condition is False
- If the condition is always True, the *while* loop will iterate endlessly
  - Ctrl-C to stop the program execution



While Loops Syntax

- A While loop
  - starts with keyword *while*
  - followed by a boolean expression which ends with a colon *:*
  - the code that is to be repeated if the condition is met is placed in an indented code block
  - once the indentation goes back one level, the body of the *while* loop ends

```
while <some sort of condition>:
    <statement 1 in the loop>
    <statement 2 in the loop>
    ...
<statement out of the loop>
```

While Loops

```
# 02-while_loop_counter.py

count = 1                    # start value
while count <= 5:            # condition
    print("Count is:", count)
    count = count + 1        # update step
```

count is a sentinel variable

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
```

While Loops

In-Class Questions 5.1

```
# 02-while_loop_counter.py

count = 1                    # start value
while count <= 5:            # condition
    print("Count is:", count)
    count = count + 1        # update step
```

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
```

Q1. *How many loops?*

Q2. *How many iterations?*

While Loops

In-Class Questions 5.2

```
# count_to_10.py

count = 1                    # start value
while count <= 10:           # condition
    print("Count is:", count)
    count = count + 1        # update step
print("Final value of count is:", count)
```

```
Count is: 1
Count is: 2
Count is: 3
Count is: 4
Count is: 5
Count is: 6
Count is: 7
Count is: 8
Count is: 9
Count is: 10
Final value of count is: ???
```

While Loops

```
# 03-while_loop_password.py

password = "ICPFALL25"
stop = False
user_input = ""

while not stop:
    user_input = input("Enter the password:\n")
    if user_input == password:
        stop = True

print("Access granted!")
```

```
>>> Enter the password:
    ICPSPRING24
>>> Enter the password:
    ICPFALL25
    Access granted!
```

Add a **maximum attempt limit** so the user only gets 3 tries before access is denied

While Loops

In-Class Questions 5.3

- Write a program that
  - prompts the user for a positive integer
  - prints a countdown from that number to zero.

```
# countdown.py

a = int(input("Enter a positive integer:> "))

while [condition]:
    print(a)
    a = [operation]
```

```
>>> Enter a postive integer:> 10
10
9
8
7
6
5
4
3
2
1
0
```

While Loop Example 1

- Write a program to count from 2 through 8 by 2's The output should be:

  2
  4
  6
  8
  done

While Loop Example 1

- Write a program to count from 2 through 8 by 2's. The output should be:

  2
  4
  6
  8
  done

```
# version 1
count = 2
while count <= 8:
  if count % 2 == 0:
     print (count)
  count = count + 1
print("done")
```

```
# version 2
count = 2
while count <= 8:
  print (count)
  count = count + 2
print("done")
```

While Loop Example 2

- Write a program that
  - converts an integer from decimal to binary
  - exits if user enters "stop"

While Loop Example 2

- Write a program that
  - converts an integer from decimal to binary
  - exits if user enters "stop"

```
stop = False

while not stop:
    a = input("Enter an integer or 'stop' to exit:> ")
    if a != 'stop':
        print(bin(int(a)))
    else:
        stop = True
```

Accumulator

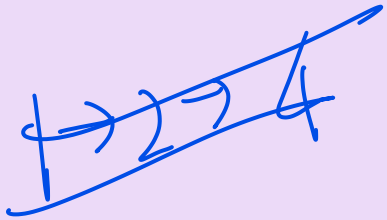
- An accumulator is a variable used in programming to collect a value over multiple iterations of a loop
  - Typically, it starts with an initial value (like 0 for sums or 1 for prods).
  - Inside the loop, you update it each time to keep a running total.

```
# 03-accumulator.py

num = 0                         # sentinel
total = 0                       # accumulator

while num < 5:
    total = total + num         # add each number to the total
    num = num + 1
print(total)                    # output: 10
```

Accumulator

In-Class Questions 5.4

```
# accumulator-1.py

num = 1
total = 1

while num < 5:
    total = total * num
    num = num + 1

print(total)
```



1 → 2 → 6 → 24 → 120.

- Sentinel variable? No
- Accumulator variable? Yes → total
- Program output? 120

Augmented Assignment Operators

- The value of a sentinel/accumulator variable is usually mutated depending on its previous value
  - i = i +1
  - n = n + 2
  - factorial = factorial * i
- Augmented assignment operators
  - +=
  - -=
  - *=
  - /=
  - %=

Augmented Assignment Operators

In-Class Questions 5.5

Operator	Equivalent to	Example usage
`+=`	`x = x + 5`	
`-=`	`y = y - 2`	
`*=`	`z = z * 10`	
`/=`	`a = a / b`	
`%=`	`c = c % 3`	

While Loop Example (3)

- Write a program that
  - print all the numbers from 1 to a max value entered by the user
  - Except these including at least one 4

Modules

Modules

- Few built-in functions are all available by default:
  - print, input, int, float, str, type, …
  - abs, len, min, …
- A module is a file that contains Python code
  - definitions and functions
    - that are not automatically loaded when you run Python
    - intended for use in other Python programs
  - the contents of a module are made available to the other program by using the *import* statement

Module Examples

math module

- Modules you can import include
  - import math
  - import random
  - import sys
- math module
  - definitions
    - *pi* a constant that contains the value of π
  - functions
    - *floor(x)* returns the smallest integer less than or equal to x
    - *ceil(x)* returns the smallest integer greater than or equal to x
    - *sqrt(x)* returns the square root of x
    - *cos(x)* returns the cosine of x (expressed in radians)
    - …

Modules

- How to call functions from modules?
  - Import the module first
  - using the module name as the prefix
  - a dot .
  - the function (or variable) name
- Examples

```
>>> import math
>>> math.pi
    3.141592653589793
>>> math.sqrt(25)
    5.0
```

Module Examples

random, sys, os modules

- random module
  - *random()* returns a float that is between 0 and 1
  - *randint(a,b)* returns a random int that is between *a* and *b* (included)
- sys module
  - *version* a constant that contains the version of Python
  - *exit()* exits from Python
- os module
  - *uname()* returns information about current operating system

Module Examples

random, sys, os modules

```
# 04-module_examples.py
import math, random, sys, os


print(math.pi)
print(random.randint(0,10))
print(sys.version)
print(os.uname())
```

```
3.141592653589793
7
3.10.0 (v3.10.0:b494f5935c, Oct  4 2021, 14:59:20) [Clang 12.0.5
(clang-1205.0.22.11)]
posix.uname_result(sysname='Darwin', nodename='MacBook-Prom.local',
release='18.7.0', version='Darwin Kernel Version 18.7.0: Tue Aug 20
16:57:14 PDT 2019; root:xnu-4903.271.2~2/RELEASE_X86_64', machine='x86_64')
```

Module Examples

In-Class Questions 5.6

```
import [module 1], [module 2]


num = [module 2].randint(0,10)
power = [module 1].pow(num,2)
print(power)
```

Takeways

- While repetition structure
  - while loops repeat a block of code as long as a condition is True
  - the number of repetitions is controlled by a counter/ sentinuel variable
  - the intermediate result is kept in an accumulator variable
- Module
  - Modules are files containing Python statements and definitions (functions and variables)
  - Python has over 200 standard modules

Readings & Assignments

- Assignments (deadline: Tue 09/30 09:45pm):
  - Quizz 04
- Preparation for Lecture 05 (10/07):
  - Read chapter 4 in the textbook with special attention to paragraph 4.3
  - Watch self-paced modules 5:
    - https://stream.nyu.edu/playlist/dedicated/306991832/1_wbvb3ll2/